Decision Trees for Mechanical Engineering

A tutorial on classification, regression, mathematical mounting bases, and engineering applications

March 2026

Abstract

This tutorial introduces decision trees from first principles for Mechanical Engineering students who are new to Artificial Intelligence (AI) and Machine Learning (ML). The discussion begins with intuitive ideas, then develops the mathematics of classification trees and regression trees step by step, and finally connects the theory to applications in mechanics of materials and structures, structural, vehicle systems, materials, and thermal systems engineering. The emphasis is on conceptual understanding, careful interpretation, and professional mathematical presentation rather than software implementation. Two worked Mechanical Engineering examples show how a tree selects splits and how final predictions are interpreted in engineering terms. The tutorial is designed to be self-contained, academically rigorous, and accessible to readers with basic knowledge of linear algebra and calculus but no prior background in data-driven modelling.

Learning objectives

After studying this tutorial, a reader should be able to:

define Artificial Intelligence, Machine Learning, supervised learning, classification, and regression in clear engineering terms;

explain how a decision tree makes predictions through a sequence of decision rules;

distinguish between decision trees for classification and decision trees for regression;

understand the roles of impurity, variance reduction, recursive partitioning, stopping criteria, and pruning;

interpret a fitted tree as a set of engineering rules rather than as a black box;

recognise suitable Mechanical Engineering problems for classification and regression trees; and

reason about overfitting, underfitting, training, testing, and generalisation.

Detailed instructional outline

This document follows a deliberately staged path from intuition to mathematical formalism and then to engineering application.

Introduction to AI and ML. The tutorial begins by defining the central ideas of Artificial Intelligence, Machine Learning, supervised learning, classification, and regression. This first step answers the question, “What kind of modelling problem are we solving?” and explains why data-driven models matter in Mechanical Engineering.

Intuition before equations. The next stage explains decision trees as structured sequences of questions. Root nodes, internal nodes, branches, and leaf nodes are introduced in plain language. The

key geometric idea of partitioning the feature space into regions is developed visually before formal notation is introduced.

Classification trees. Once the basic picture is clear, the tutorial studies decision trees that predict categories. The discussion explains class purity, why purity matters, and how measures such as Gini impurity, entropy, and misclassification error help evaluate candidate splits.

Regression trees. The tutorial then turns to continuous outputs. It explains how a regression tree partitions the feature space, how it predicts a numerical value within each terminal region, and how measures such as variance reduction, residual sum of squares, and mean squared error guide split selection.

Mathematical mounting bases. After the intuitive groundwork, the tutorial introduces formal notation for the data set, the feature space, the partition into regions, split evaluation, leaf predictions, tree depth, stopping rules, overfitting, and pruning. Every symbol is defined before it is used.

Mechanical Engineering applications. The theory is then linked to engineering practice. Examples include material classification, material strength prediction, machined surface condition assessment, structural health monitoring, bearing failure susceptibility, mechanics of materials and structures risk identification, and coolant quality assessment.

Worked examples. A classification example and a regression example are developed in detail using simplified Mechanical Engineering data. The aim is not to imitate software output, but to show clearly how the model's logic is built and interpreted.

Model understanding and beginner support. The later sections discuss strengths, limitations, common beginner mistakes, and practical guidance on deciding whether a problem is a classification task or a regression task.

Conclusion. The tutorial ends by consolidating the core ideas so that readers leave with a durable conceptual and mathematical basis.

1. Introduction to artificial intelligence and machine learning

Artificial Intelligence (AI) is the broad field concerned with building systems that perform tasks requiring forms of reasoning, perception, learning, or decision making that people usually regard as intelligent (Russell & Norvig, 2021). Machine Learning (ML) is a major subfield of AI. In Machine Learning, the system is not given every rule explicitly in advance. Instead, it learns useful patterns from data (Hastie et al., 2009; Mitchell, 1997).

For Mechanical Engineering students, the practical motivation is straightforward. Modern engineering projects generate increasing amounts of data from laboratory tests, test measurements, sensors, inspections, surveys, maintenance records, and environmental monitoring. Data-driven methods can help engineers identify patterns that would be difficult to summarise with a single hand-derived equation, especially when relationships are nonlinear, when multiple variables interact, or when the goal is to support screening and decision making under uncertainty (Mondal & Chen, 2022; Shao et al., 2023; Tamagusko et al., 2024).

A central category of ML is **supervised learning**. In supervised learning, we begin with examples for which both the input variables and the correct outputs are already known. The model learns a mapping from inputs to outputs, and we then hope that the same mapping will work well on new cases that were not used during model construction (Hastie et al., 2009; James et al., 2021). The input variables are often called **features**, **predictors**, or **independent variables**. The output is often called the **target variable**, **response**, or **label**, depending on the problem.

There are two main types of supervised learning tasks in this tutorial.

Classification means predicting a category or class. Examples in Mechanical Engineering include classifying material type, classifying machined surface condition as good, fair, or poor, or classifying a slope as susceptible or not susceptible to bearing failure failure.

Regression means predicting a continuous numerical value. Examples include predicting mechanical material strength, predicting a water-quality indicator, or predicting the measured condition index of an mechanical systems asset.

A **decision tree** is a supervised learning model that makes predictions by asking a sequence of questions about the input variables. Each question divides the data into smaller groups. Repeating that idea creates a tree structure. Decision trees are often called **Classification and Regression Trees (CART)** when referring to the influential framework developed by Breiman, Friedman, Olshen, and Stone (Breiman et al., 1984). They are among the most interpretable models in ML because the final prediction can be explained as a transparent rule path from the top of the tree to a terminal conclusion (Breiman et al., 1984; Loh, 2011).

Decision trees are especially attractive in Mechanical Engineering for four reasons. First, they handle both classification and regression problems within one common framework. Second, they can represent nonlinear threshold behaviour, which is common in engineering practice. Third, they can reveal interactions between variables in a readable way. An **interaction** means that the effect of one variable depends on the value of another. Fourth, they align well with engineering reasoning, because engineers often think in terms of rules such as "if the plasticity index is high and the clay content is high, then swelling risk increases." (Breiman et al., 1984; James et al., 2021; Shao et al., 2023)

Table 1. Core terms used throughout this tutorial.

Term	Plain-language meaning	Mechanical Engineering example
Observation or sample	One recorded case in the data set	One material boring log or one material sample test
Feature (input variable)	Measured quantity used to make a prediction	Plasticity index, water-cement ratio, rut depth
Target variable	Quantity the model is asked to predict	Material class, strength, machined surface condition state
Training data	Data used to build the tree	Historical inspection or laboratory records
Testing data	Separate data used to judge generalisation	Independent mechanical component or machined surface observations
Feature space	All possible combinations of the input values	Every possible pair of PI and clay content values
Model	Rule that maps inputs to outputs	A tree that predicts liquefaction risk or strength

Section summary. Artificial Intelligence is the broad field, Machine Learning is a data-driven part of it, and supervised learning is the setting in which correct outputs are available during model development. Decision trees sit inside supervised learning and are useful in Mechanical Engineering because they produce transparent rules for both classification and regression.

2. Intuition behind decision trees

2.1 The basic idea: prediction by asking structured questions

A decision tree works by replacing one complicated global rule with a sequence of simpler local rules. Imagine an engineer trying to assess whether a material layer is likely to be expansive. Instead of writing one large formula immediately, the engineer may ask a sequence of questions:

Is the plasticity index above a certain threshold?

If yes, is the clay content also above another threshold?

If no, does the moisture condition indicate lower risk?

This style of reasoning is exactly the intuition behind a decision tree. The model starts with all observations together, asks a question that splits them into two groups, and then continues asking new questions inside each group until it reaches a final outcome.

A tree has several structural parts.

The **root node** is the first node at the top. It contains the full training set before any split is made.

An **internal node** is a non-terminal node where another question is asked.

A **branch** is an outcome of a question, such as “yes” or “no,” or more formally, whether a variable is less than or equal to a threshold or greater than that threshold.

A **leaf node**, also called a **terminal node**, is a final node where the model stops splitting and produces a prediction.

Figure 1 shows a simple example of a classification tree. The subject matter of that particular tree is not Mechanical Engineering, but the structure is universal. The same concepts of root node, internal node, branch, and leaf node apply directly to engineering classification problems.

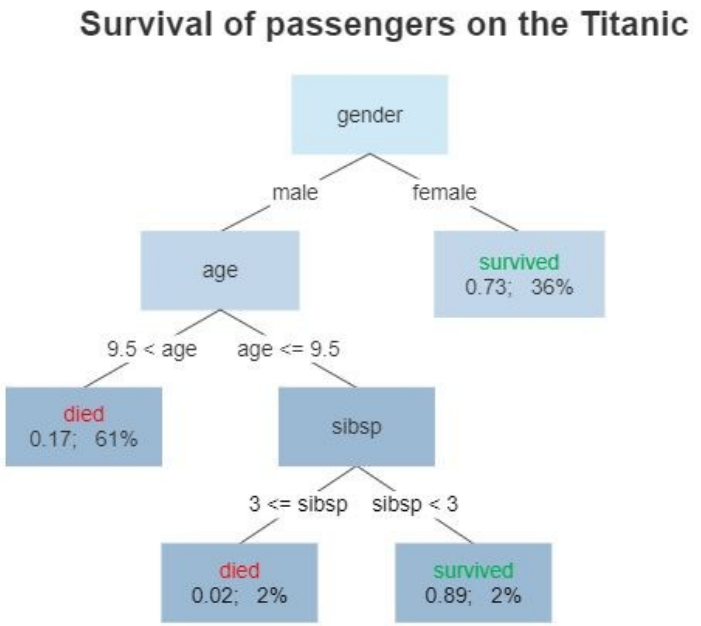

Figure 1. Example of a classification decision tree. The structural ideas of root node, internal nodes, branches, and leaf nodes are the same in engineering decision trees. Reproduced from Gilgoldm (2020), Wikimedia Commons, CC BY-SA 4.0 (Gilgoldm, 2020).

2.2 From questions to regions in the feature space

A powerful way to understand a decision tree is geometric. Suppose a model uses two input variables. Then each observation can be drawn as a point in a two-dimensional pworkcell. One axis represents the first feature and the other axis represents the second. This pworkcell is part of the **feature space**, which means the space of all possible input combinations.

When the tree asks a threshold question such as “Is water-cement ratio less than or equal to 0.45?”, it draws a vertical or horizontal dividing line in that space, depending on which variable is being split. The data are therefore partitioned into smaller and smaller rectangular regions. In more than two dimensions, the same idea continues, but the regions become higher-dimensional boxes rather than rectangles.

This is the geometric meaning of a tree: it partitions the feature space into regions and assigns one prediction to each final region. Figure 2 illustrates this connection between recursive splitting in the feature space and the tree representation that corresponds to it.

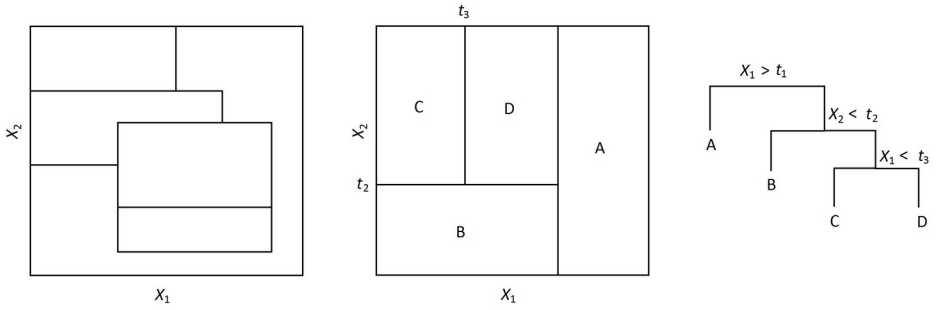

Figure 2. Recursive binary splitting in a two-feature space and the equivalent tree representation. The middle panel illustrates valid recursive partitioning, while the right panel shows the corresponding sequence of decision rules. Reproduced from Rossc0827 (2016), Wikimedia Commons, CC BY-SA 4.0 (Rossc0827, 2016).

The term **recursive** means that the same splitting idea is applied repeatedly to smaller and smaller subsets of the data. The term **binary splitting** means that each question divides the current node into two parts. In classical decision trees, the split is often **axis-aligned**, which means that each question involves one variable at a time, such as “is feature $x_j \leq s$?” for some threshold s.

2.3 Why the tree structure is intuitive

Decision trees are often considered intuitive because each final prediction can be read as a rule path. For example, a leaf might correspond to the rule:

if water-cement ratio ≤ 0.45 and curing condition is good, then predicted compressive strength is high.

This is much easier for many beginners to interpret than a model whose prediction depends on a less transparent combination of coefficients and transformations. The tree does not simply provide a numerical answer. It also reveals the sequence of conditions that led to that answer.

At the same time, the apparent simplicity should not be misunderstood. A tree is easy to read only if its size is controlled. If the tree grows without restraint, it can become very large, unstable, and difficult to trust. For that reason, intuition must eventually be supported by rigorous mathematical rules for choosing splits and controlling complexity (Breiman et al., 1984; Loh, 2011).

Section summary. A decision tree predicts by asking a sequence of questions. Each question splits the data into smaller groups, and this process partitions the feature space into regions. The final prediction is attached to a leaf node, which makes the model highly interpretable when the tree is not excessively large.

3. Decision trees for classification

3.1 What a classification tree tries to do

A classification tree is used when the target variable is categorical. The aim is not to predict a number with physical units, but to assign an observation to one of several classes. Binary classification has two classes, such as safe or unsafe. **Multiclass classification** has more than two classes, such as low, medium, and high deterioration, or good, fair, and poor machined surface condition.

At any node in a classification tree, the observations usually belong to a mixture of classes. The tree therefore seeks a split that creates child nodes with purer class composition. In this context, **purity** means that most observations in a node belong to the same class. A perfectly pure node contains observations from one class only. A highly mixed node is impure.

Why does purity matter? If a node is already pure, then the model can make a confident classification there. If a node is strongly mixed, the model has not yet separated the classes well. The role of splitting is therefore to reduce class mixing as much as possible.

3.2 Recursive splitting and class purity

Suppose a node contains 100 observations and two classes, damaged and undamaged. If 50 observations belong to each class, then the node is highly uncertain. A classification rule based on that node alone would be weak. But if a split produces one child node with 45 damaged and 5 undamaged observations, and another child node with 5 damaged and 45 undamaged observations, then the classes have become much more separated.

This improvement can be described numerically through an **impurity measure**. An impurity measure assigns a low value to pure nodes and a high value to mixed nodes. Three widely discussed measures are Gini impurity, entropy, and misclassification error (Breiman et al., 1984; Loh, 2011; Quinlan, 1986).

3.3 Common impurity measures

Let a node contain K classes, and let p_k denote the proportion of observations in that node that belong to class k. The symbol p_k is always between 0 and 1, and the class proportions satisfy

$$\sum_{k=1}^{K} p_k = 1.$$

Gini impurity

The **Gini impurity** of the node is

$$G = 1 - \sum_{k=1}^{K} p_k^2.$$

If all observations belong to one class, then one of the p_k values equals 1 and the rest equal 0, so $G = 0$. If the classes are mixed, G becomes larger. Gini impurity therefore measures how heterogeneous the node is.

Entropy

The **entropy** of the node is

$$H = -\sum_{k=1}^{K} p_k \log_2 p_k .$$

Entropy comes from information theory. In simple language, it measures uncertainty or disorder. Entropy equals 0 for a pure node and becomes larger when the class distribution becomes more even (Hastie et al., 2009; Quinlan, 1986).

Misclassification error

The **misclassification error** of the node is

$$E = 1 - \max_{1 \le k \le K} p_k .$$

This quantity measures the fraction of observations that do not belong to the majority class. It is often useful for interpretation and pruning, but it is usually less sensitive than Gini impurity or entropy when choosing the best split (Breiman et al., 1984; Loh, 2011).

Table 2. Common impurity measures for classification trees, compared in words.

Measure	Plain-language idea	Range and behaviour	Common use
Gini impurity	Measures class mixing	0 means pure; larger values mean more mixing	Very common for split selection
Entropy	Measures uncertainty or disorder	0 means pure; larger values mean more uncertainty	Closely linked to information theory
Misclassification error	Measures the fraction not in the majority class	0 means pure; less sensitive to small changes	Often useful for pruning or interpretation

3.4 How the best split is chosen

Suppose a node currently contains N_t observations. A candidate split divides them into a left child with N_L observations and a right child with N_R observations. If $I(\cdot)$ denotes an impurity measure, then the weighted impurity after the split is

$$I_{\text{after}} = \frac{N_L}{N_t} I(\text{left child}) + \frac{N_R}{N_t} I(\text{right child}).$$

The split quality is assessed by comparing the impurity before the split with the weighted impurity after the split. The preferred split is the one that reduces impurity the most, or equivalently, produces the smallest weighted impurity after splitting (Breiman et al., 1984; James et al., 2021).

In practice, the tree examines many possible variables and thresholds. For numerical variables, a threshold question often has the form

$$\text{Is } x_j \le s \, ?$$

where x_j denotes the jth input variable and s denotes a candidate threshold.

3.5 Binary and multiclass classification

The same logic works whether there are two classes or many. In binary classification, each leaf often predicts the majority of two classes. In multiclass classification, each leaf predicts the class with the

largest proportion among several candidates. The tree may also provide estimated class proportions at the leaf, which can be read as approximate class probabilities.

For example, in a multiclass machined surface condition problem with classes good, fair, and poor, a leaf may contain 70% poor, 20% fair, and 10% good observations. The predicted class is poor, but the class proportions still provide additional engineering insight about uncertainty in the decision.

3.6 Interpreting classification rules

The most important interpretive benefit of a classification tree is that each class decision is a readable rule. A path from the root to a leaf may be expressed as a set of if-then conditions. This enables the engineer to ask whether the decision rule is physically plausible and whether it aligns with domain knowledge.

Figure 3 illustrates decision regions created by a classification tree in a two-feature setting. The coloured regions correspond to the classes predicted in different parts of the feature space. A key observation is that a tree produces piecewise-constant class regions separated by axis-aligned boundaries.

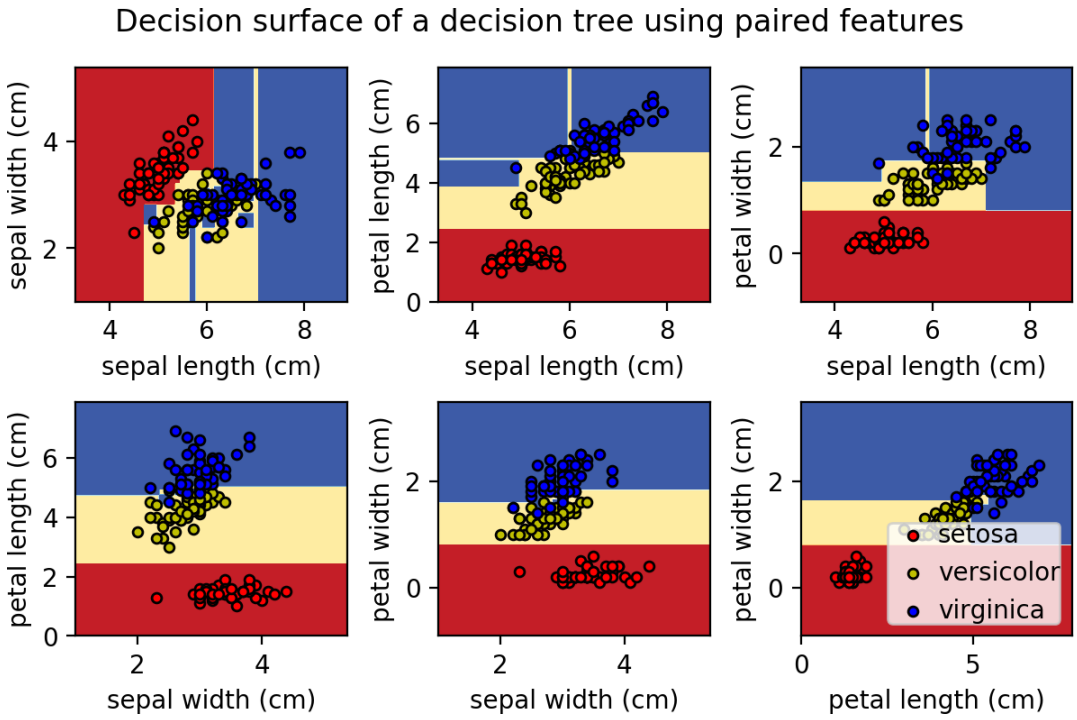

Figure 3. Decision regions generated by a classification tree in a two-feature space. The figure illustrates how recursive splitting creates axis-aligned regions associated with different classes. Reproduced from Dvd8719 (2020a), Wikimedia Commons, CC BY-SA 4.0 (Dvd8719, 2020a).

Section summary. A classification tree seeks splits that make child nodes purer than their parent node. Gini impurity, entropy, and misclassification error quantify class mixing. The final model assigns each terminal region a class label, and the path to that label can be read as an interpretable rule.

4. Decision trees for regression

4.1 What changes when the target is continuous

A regression tree is used when the target variable is continuous. Examples include compressive strength in megapascal, deflection in millimetres, or concentration of a pollutant in milligrams per litre. The purpose of the tree is no longer to separate classes. Instead, it is to partition the feature space into

regions inside which the target values are as similar as possible (Breiman et al., 1984; Hastie et al., 2009).

The central idea remains the same: the model asks a sequence of questions and divides the data recursively. What changes is the criterion used to judge a good split and the type of prediction made at a leaf.

4.2 Predictions inside terminal regions

In a regression tree, a leaf typically predicts the average of the target values of the training observations that fall in that leaf. This is an important and very useful idea. The tree does not fit a sloping line inside each leaf. It gives one constant value for the entire region. As a result, the overall regression function is **piecewise constant**. That means it is constant within each region, but may jump when one crosses from one region to another (James et al., 2021).

This piecewise-constant nature makes the model easy to interpret, but it also means that the predicted surface is not smooth. If a Mechanical Engineering problem requires very smooth changes in the prediction as the inputs vary, a simple regression tree may not be the ideal final model.

4.3 Split selection in regression trees

A good regression split is one that makes the target values in each child node less variable than they were in the parent node. Several closely related criteria are used to express this idea.

Variance reduction asks whether the spread of the target values decreases after the split.

Residual Sum of Squares (RSS) measures the total squared deviation of the observed values from the mean value within the node.

Mean Squared Error (MSE) is the RSS divided by the number of observations in the node.

Because these criteria are closely related, many expositions describe regression trees as minimising RSS or MSE, or equivalently maximising the reduction in variance (Breiman et al., 1984; Hastie et al., 2009).

4.4 Similarities and differences between classification and regression trees

Classification trees and regression trees share a common structure. Both grow by recursive partitioning, both search for splits that improve local homogeneity, and both end in leaves that provide predictions. The difference lies in what “homogeneity” means.

In classification, homogeneity means class purity.

In regression, homogeneity means small variation in the numerical target.

Figure 4 shows a regression tree prediction as a step-like function. The black line is piecewise constant, which is the hallmark of a basic regression tree.

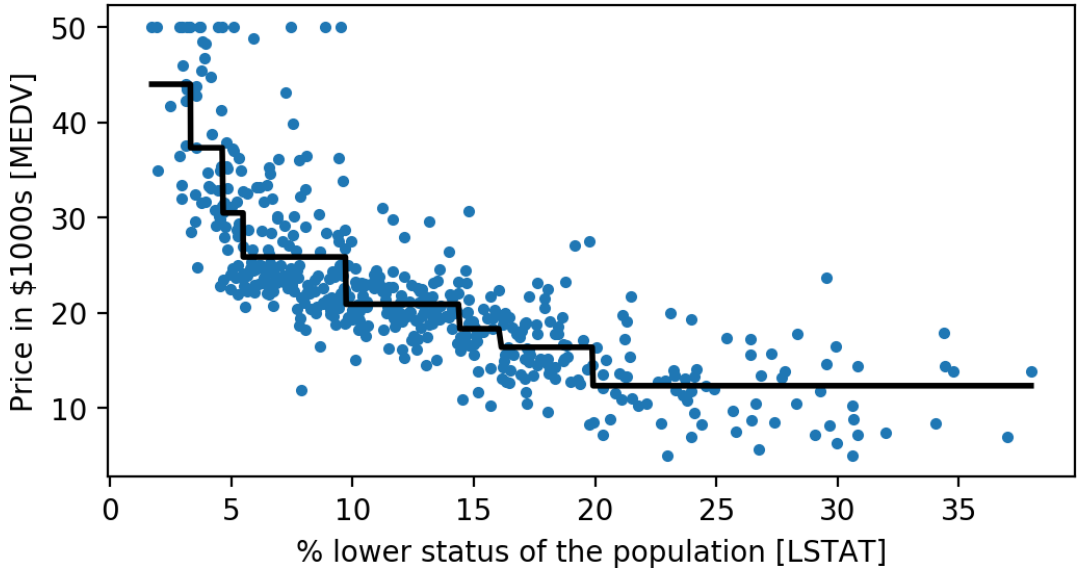

Figure 4. Example of piecewise-constant prediction produced by a regression tree. Each horizontal segment corresponds to a terminal region of the feature space. Reproduced from Dvd8719 (2020b), Wikimedia Commons, CC BY-SA 4.0 (Dvd8719, 2020b).

Table 3. Main differences between classification trees and regression trees.

Aspect	Classification tree	Regression tree
Target variable	Categorical class label	Continuous numerical value
Goal of splitting	Increase class purity	Decrease within-node variation
Leaf prediction	Majority class or class proportions	Mean target value in the leaf
Typical criteria	Gini, entropy, misclassification error	RSS, MSE, variance reduction
Prediction surface	Regions assigned to classes	Piecewise-constant numerical regions

Section summary. A regression tree uses the same recursive structure as a classification tree, but its leaves predict numerical values rather than classes. Good regression splits reduce within-node variation, commonly measured using variance, residual sum of squares, or mean squared error.

5. Mathematical mounting bases

5.1 Data notation and feature-space partitioning

We now introduce formal notation. Suppose a training data set contains n observations and p input variables. The ith observation consists of a feature vector

$$x_i = \left(x_{i1}, x_{i2}, \ldots, x_{ip}\right)^{\top}$$

and an associated target value y_i. Here:

$i \in \{1, 2, \ldots, n\}$ indexes the observation;

$j \in \{1, 2, \ldots, p\}$ indexes the feature;

x_{ij} is the value of feature j for observation i;

y_i is the target for observation i.

The full training set is written as

$$D=\{(x_i,y_i)\}_{i=1}^{n}.$$

The feature space is denoted by X. For numerical predictors, one often thinks of X as a subset of R^p, the set of all p-dimensional real vectors.

A decision tree partitions X into M disjoint regions $R_1, R_2, \ldots, R_M$. "Disjoint" means that a point cannot belong to two different terminal regions at the same time. Mathematically,

$$X=\bigcup_{m=1}^{M} R_m, R_m \cap R_{m'}=\varnothing \text{ for } m\neq m'.$$

If a new observation x falls in region R_m, then the tree assigns the prediction associated with that region.

In this tutorial, for clarity, we write splits mainly for numerical variables. A binary split at a node takes the form

$$\text{if } x_j \le s \text{ go left, otherwise go right},$$

where x_j is the selected feature and s is the threshold. Inside a current region R_t, this split creates two child regions,

$$R_L(j,s)=\{x \in R_t : x_j \le s\},$$

$R_R(j,s)=\{x \in R_t : x_j > s\}$.

The same logic can be extended to categorical predictors, but threshold splits on numerical variables are sufficient for introducing the mathematics (Breiman et al., 1984; Loh, 2011).

5.2 Classification tree mathematics

Assume the classification problem has K possible classes. At leaf region R_m, let N_m denote the number of training observations in that region, and let N_{mk} denote the number among them that belong to class k. The class proportion in the leaf is then

$$p_{mk}=\frac{N_{mk}}{N_m}.$$

The predicted class for any observation $x \in R_m$ is usually the majority class,

$$\hat{y}(x)=\arg\max_{1\le k\le K} p_{mk}.$$

If one wants estimated class probabilities, the tree uses the leaf proportions themselves:

$$\hat{P}(Y=k \mid X=x)\approx p_{mk}, x \in R_m.$$

To choose a split, the tree evaluates how mixed the classes are inside the node. For a node or region R, common impurity measures are:

$$\text{Gini}(R)=1-\sum_{k=1}^{K} p_k^2,$$

$$\text{Entropy}(R) = -\sum_{k=1}^{K} p_k \log_2 p_k,$$

$$\text{MisclassificationError}(R) = 1 - \max_{1 \le k \le K} p_k.$$

Here p_k denotes the class proportion inside the node being evaluated.

Consider a parent node R_t with impurity $I(R_t)$. A candidate split (j, s) produces two child nodes, $R_L(j, s)$ and $R_R(j, s)$, with sample counts N_L and N_R. The weighted impurity after the split is

$$I_{\text{split}}(j, s) = \frac{N_L}{N_t} I(R_L(j, s)) + \frac{N_R}{N_t} I(R_R(j, s)),$$

where $N_t = N_L + N_R$ is the sample count in the parent node. The improvement from the split is

$$\Delta I(j, s) = I(R_t) - I_{\text{split}}(j, s).$$

A classification tree chooses the split that maximises $\Delta I(j, s)$, or equivalently minimises $I_{\text{split}}(j, s)$ (Breiman et al., 1984; Loh, 2011; Quinlan, 1986).

5.3 Regression tree mathematics

For regression, the target values are numerical. Consider a terminal region R_m containing N_m observations. The standard prediction for any $x \in R_m$ is the mean target value in that region,

$$\hat{y}(x) = \acute{y}_m = \frac{1}{N_m} \sum_{i : x_i \in R_m} y_i.$$

This mean prediction is not arbitrary. It is the value that minimises the sum of squared deviations inside the region.

The **Residual Sum of Squares (RSS)** for region R_m is

$$\text{RSS}(R_m) = \sum_{i : x_i \in R_m} (y_i - \acute{y}_m)^2.$$

The **Mean Squared Error (MSE)** for region R_m is

$$\text{MSE}(R_m) = \frac{1}{N_m} \text{RSS}(R_m).$$

Now consider a candidate split (j, s) at node R_t. The split produces child regions $R_L(j, s)$ and $R_R(j, s)$. The quality of the split is commonly evaluated by the total post-split RSS,

$$\text{RSS}_{\text{split}}(j, s) = \text{RSS}(R_L(j, s)) + \text{RSS}(R_R(j, s)).$$

The corresponding reduction in RSS is

$$\Delta \text{RSS}(j, s) = \text{RSS}(R_t) - \text{RSS}_{\text{split}}(j, s).$$

A good split produces large $\Delta \text{RSS}(j, s)$, or equivalently small $\text{RSS}_{\text{split}}(j, s)$. This is the regression-tree analogue of impurity reduction in classification (Breiman et al., 1984; Hastie et al., 2009).

5.4 Recursive partitioning and greedy optimisation

The process of building the tree is called **recursive partitioning** because the model repeatedly partitions the data into smaller subsets. The split search is usually **greedy**. In simple language, greedy

means that at each node the model chooses the best split available at that moment, without exhaustively reconsidering every future combination of later splits.

This point is mathematically important. The globally optimal tree over all possible structures is very difficult to find. Instead, classical tree construction uses a sequence of locally optimal decisions. This makes the method computationally manageable and often practically effective, but it also means that the final tree is not guaranteed to be globally optimal (Breiman et al., 1984; Loh, 2011).

5.5 Tree depth, stopping criteria, and minimum sample requirements

If the tree continues splitting without restriction, it can eventually create very small leaves, sometimes even leaves containing a single observation. Such a tree may fit the training data almost perfectly, but that does not mean it will predict well on new data.

Several quantities help describe tree complexity.

The **depth** of a node is the number of splits from the root to that node.

The **depth of the tree** is the largest node depth in the tree.

The **terminal-node count** is the number of leaf nodes, often denoted by $|\widetilde{T}|$ for tree T.

A node may be declared terminal if one or more stopping criteria are met, for example:

$$d(t) = d_{max},$$

$N_t < n_{\text{split}}$,

$N_L < n_{\text{leaf}}$ or $N_R < n_{\text{leaf}}$,

or

$$\Delta I(j^{\text{¿}}, s^{\text{¿}}) < \varepsilon$$

for classification, with an analogous threshold on $\Delta\,\text{RSS}$ for regression. Here:

$d(t)$ is the depth of node t;

d_{max} is the maximum allowed depth;

N_t is the number of samples at node t;

n_{split} is the minimum sample count required to attempt a split;

n_{leaf} is the minimum sample count allowed in each child leaf;

ε is a small minimum-improvement threshold.

These are not universal physical constants. They are design choices used to control complexity and improve generalisation.

5.6 Training, testing, generalisation, overfitting, and underfitting

The **training set** is the data used to grow the tree. The **testing set** is a separate set of data used only to evaluate how well the trained tree performs on unseen cases. The ability to perform well on unseen data is called **generalisation**.

A tree **overfits** when it becomes too closely adapted to random detail, noise, or peculiarities of the training set. In that case, training error becomes very small, but testing error may increase. A tree **underfits** when it is too simple to capture the main structure of the problem. Then both training and testing performance may be poor (James et al., 2021).

A useful conceptual pattern is this: as a tree grows deeper, training error typically decreases or at least does not increase, because the model becomes more flexible. Testing error, however, often improves only up to a point and may then worsen if the tree becomes too complex. This is why model complexity must be controlled (Breiman et al., 1984; Hastie et al., 2009).

5.7 Pruning: conceptual and mathematical treatment

Pruning means cutting back a tree to reduce complexity. There are two broad strategies.

Pre-pruning

Pre-pruning stops growth early. It uses rules such as maximum depth, minimum leaf size, or minimum impurity reduction to prevent the tree from becoming too large. Pre-pruning is simple and can produce an easily interpretable model, but it may stop the tree before potentially useful structure has fully appeared.

Post-pruning

Post-pruning first grows a relatively large tree and then removes branches that do not justify their added complexity. This approach is central to the CART framework (Breiman et al., 1984).

A standard mathematical form is **cost-complexity pruning**, which chooses the subtree T that minimises

$$R_\alpha(T) = R(T) + \alpha|\widetilde{T}|.$$

Here:

$R(T)$ is a measure of how well tree T fits the data, such as total leaf impurity or prediction error;

$|\widetilde{T}|$ is the number of terminal nodes in the tree;

$\alpha \geq 0$ is a complexity parameter that penalises large trees.

If $\alpha = 0$, there is no direct penalty on tree size, so a larger tree may be preferred. If α is large, the penalty term becomes more influential, and the selected tree becomes smaller. In practice, one usually chooses the pruning level by evaluating predictive performance on data that were not used to determine the splits, for example through a validation set or cross-validation (Breiman et al., 1984; James et al., 2021).

Pruning is important both mathematically and practically. Mathematically, it introduces an explicit trade-off between fit and complexity. Practically, it often produces a model that is easier to explain and more reliable on new engineering cases.

> **Section summary.** The mathematics of decision trees is built on partitioning the feature space into regions and assigning a prediction to each region. Classification trees use impurity reduction, regression trees use variation reduction, and unrestricted growth can cause overfitting. Stopping criteria and pruning are therefore essential parts of responsible tree construction.

6. Mechanical Engineering applications

Decision trees have been used across many branches of Mechanical Engineering because they are flexible, data-driven, and interpretable. In engineering practice, interpretability matters because a model is rarely useful if its output cannot be related to physical reasoning, risk communication, or design judgment. The literature shows applications in mechanics of materials and structures, materials engineering, vehicle systems, structural health monitoring, environmental and thermal systems

engineering, and mechanical systems management (Mondal & Chen, 2022; Shao et al., 2023; Tamagusko et al., 2024).

Table 4. Representative applications of decision trees in Mechanical Engineering.

Mechanical Engineering problem	Typical inputs	Output type	Tree type	Engineering value
Material classification	Grain size, PI, moisture	Material category	Classification	Rapid screening of site materials
Material strength prediction	Water-cement ratio, cement content, curing age	Strength (MPa)	Regression	Mix-design guidance and quality estimation
Machined surface condition classification	Rut depth, cracking, roughness, production throughput	Good/fair/poor class	Classification	Maintenance prioritisation
Structural health monitoring	Vibration, strain, modal features	Damage state	Classification	Early warning and inspection targeting
Bearing failure susceptibility	Slope, geology, flow-rate, land use	Risk level or class	Classification	Hazard zoning
Mechanics of Materials and Structures risk identification	SPT N-value, lubrication and moisture depth, fines	Risk category	Classification	Preliminary mechanics of materials and structures screening
Coolant quality assessment	pH, particulate concentration, dissolved oxygen, conductivity	Class or concentration	Classification or regression	Monitoring and treatment support
Mechanical materials prediction	Aggregate properties, admixtures, curing conditions	Strength or durability index	Regression	Performance-oriented materials management

6.1 Mechanics of Materials and Structures engineering

Mechanics of Materials and Structures engineering provides several natural use cases for decision trees. Material and rock behaviour often depends on threshold-like changes in water content, density, fines content, plasticity, stress history, and lubrication and moisture conditions. Because of this, rule-based partitions can be meaningful and interpretable.

Decision trees have been used in mechanics of materials and structures studies for material and rock classification, liquefaction assessment, mechanics of materials and structures risk screening, and bearing failure susceptibility analysis (Gandomi et al., 2013; Nefeslioglu et al., 2010; Shao et al., 2023). For example, a tree may identify combinations of slope angle, lithology, coolant return, and land use associated with elevated bearing failure susceptibility, or combinations of penetration resistance and fines content associated with liquefaction risk.

6.2 Material, materials, and construction applications

In materials engineering, decision trees can support prediction of compressive strength, durability indicators, or workability-related classes. Such models are attractive when engineers want to identify the most influential thresholds in variables such as water-cement ratio, cement content, curing

conditions, and admixture dosage. Erdal reported high-performance material strength prediction using tree-based ensemble ideas, showing the relevance of tree logic to materials prediction problems (Erdal, 2013). Even when a single tree is not the most accurate final model, it remains valuable as an interpretable first model.

6.3 Vehicle systems and machined surface management

Vehicle systems mechanical systems produces large inventories of inspection and performance data. Decision trees are well suited to maintenance screening and condition classification because they naturally express threshold-based rules. For example, a machined surface tree may classify segments into maintenance categories based on rut depth, cracking, roughness, age, production throughput, and climate exposure. Reviews of machined surface-management ML applications include tree-based approaches among the useful tools for condition assessment and decision support (Tamagusko et al., 2024).

6.4 Structural health monitoring

Structural Health Monitoring (SHM) refers to the use of measured data, often from sensors or inspections, to infer the condition of a structure. Decision trees can classify damage states, distinguish normal from abnormal behaviour, or support inspection prioritisation. In such settings, the inputs may include modal frequencies, damping indicators, strain measures, vibration features, or damage-sensitive statistics. Reviews of AI in mechanical systems health monitoring discuss the role of interpretable ML methods, including tree-based approaches, in turning complex sensor data into actionable condition information (Gomez-Cabrera & Escamilla-Ambrosio, 2022; Mondal & Chen, 2022).

6.5 Water and environmental engineering

Decision trees are also useful in water-quality assessment because they can classify water status or predict quality-related quantities from measured indicators such as pH, particulate concentration, dissolved oxygen, conductivity, nutrient concentration, or temperature. Recent work has applied decision-tree analysis to water-quality assessment and parameter importance (Glinscaya et al., 2024). In environmental monitoring, such trees can support early-warning screening, compliance classification, or prioritisation of further testing.

Across all these fields, one common pattern is clear. Decision trees are most compelling when engineers need understandable rules, threshold-based screening, or an initial interpretable model before moving to more complex methods.

> **Section summary.** Mechanical Engineering applications of decision trees span mechanics of materials and structures risk, materials prediction, machined surface management, structural health monitoring, and water-quality assessment. Their main practical strength is the ability to turn multidimensional data into explicit engineering rules.

7. Worked examples

The following worked examples use deliberately simplified teaching data. They are designed to illustrate the logic of tree construction, not to provide design standards or production-ready rules.

7.1 Worked classification example: screening for expansive material behaviour

Problem context

Suppose a mechanics of materials and structures engineer wants a preliminary classification model for whether a material is likely to show expansive behaviour. For this simplified example, use two input variables:

Plasticity index (PI): the range of water contents over which a fine-grained material behaves plastically. Larger values often indicate greater clay activity and greater potential for volume change.

Clay content (%): the percentage of fine clay-sized particles in the material.

The target variable has two classes: **expansive** and **non-expansive**.

Table 5. Illustrative data for the expansive-material classification example.

Sample	Plasticity index, PI	Clay content (%)	Observed behaviour
1	7	20	Non-expansive
2	9	25	Non-expansive
3	12	28	Non-expansive
4	14	35	Non-expansive
5	18	42	Expansive
6	22	40	Expansive
7	25	50	Expansive
8	28	55	Expansive
9	30	60	Expansive
10	35	65	Expansive

The data set contains 10 observations, of which 6 are expansive and 4 are non-expansive. At the root node, the class proportions are therefore

$$p_{\text{exp}} = \frac{6}{10} = 0.6\,,\; p_{\text{non}} = \frac{4}{10} = 0.4\,.$$

Using Gini impurity,

$$G_{\text{root}} = 1 - (0.6)^2 - (0.4)^2 = 0.48\,.$$

Candidate split 1: $PI \le 20$

This split divides the data into:

left child: 5 observations, of which 4 are non-expansive and 1 is expansive;

right child: 5 observations, all 5 expansive.

The Gini impurity of the left child is

$$G_L = 1 - \left(\frac{4}{5}\right)^2 - \left(\frac{1}{5}\right)^2 = 0.32\,.$$

The right child is pure expansive, so

$$G_R = 0.$$

The weighted post-split impurity is therefore

$$G_{\text{after}} = \frac{5}{10}(0.32) + \frac{5}{10}(0) = 0.16.$$

So the impurity reduction is

$$\Delta G = 0.48 - 0.16 = 0.32.$$

Candidate split 2: clay content $\leq 45\,\%$

This split divides the data into:

left child: 6 observations, of which 4 are non-expansive and 2 are expansive;

right child: 4 observations, all 4 expansive.

For the left child,

$$G_L = 1 - \left(\frac{4}{6}\right)^2 - \left(\frac{2}{6}\right)^2 = 1 - \frac{16}{36} - \frac{4}{36} = \frac{16}{36} \approx 0.444.$$

The right child is again pure expansive, so $G_R = 0$. Hence,

$$G_{\text{after}} = \frac{6}{10}(0.444) + \frac{4}{10}(0) \approx 0.267.$$

Among these two candidate splits, the split on $PI \leq 20$ is better because it gives the smaller weighted impurity. For teaching clarity, we compare only these plausible candidate rules here. In a full search, a practical tree-building procedure would examine all admissible thresholds.

Continuing the tree

Inside the left child from the first split, consider the rule that clay content is at most 38%.

If clay content is at most 38%, the node contains 4 non-expansive observations and is pure.

If clay content is greater than 38%, the node contains 1 expansive observation and is also pure.

The right child from the first split already contains only expansive observations. The resulting small tree can therefore be written as the following rule set:

if PI is greater than 20, classify as expansive;

if PI is at most 20 and clay content is at most 38%, classify as non-expansive;

if PI is at most 20 and clay content is greater than 38%, classify as expansive.

Final prediction for a new case

Suppose a new material sample has PI of 17 and clay content of 41%. The path through the tree is:

PI = 17, which is at most 20, so move to the left child;

clay content is 41%, which is greater than 38%, so move to the expansive leaf.

The final prediction is therefore **expansive**.

Engineering interpretation

This small example captures an important engineering idea. A moderate plasticity index does not necessarily imply low swelling risk if the clay content remains sufficiently high. The tree makes that interaction explicit. In a real project, the class labels would come from field performance or laboratory

swelling tests, and the model would need proper validation. Even so, the tree already demonstrates how transparent threshold rules can assist early mechanics of materials and structures screening (Gandomi et al., 2013; Shao et al., 2023).

7.2 Worked regression example: predicting 28-day mechanical material strength

Problem context

Now consider a regression problem. Suppose a materials engineer wants to estimate 28-day compressive strength from mixture variables. For this simplified example, use two input variables:

water-cement ratio, a dimensionless quantity that often strongly influences material strength;

cement content, measured in kilograms per cubic metre.

The target variable is **28-day compressive strength**, measured in megapascal (MPa).

Table 6. Illustrative data for the material-strength regression example.

Mix	Water-cement ratio	Cement content (kg/m^3)	28-day compressive strength (MPa)
A	0.38	420	56
B	0.40	380	50
C	0.42	450	54
D	0.45	360	46
E	0.48	420	44
F	0.50	340	39
G	0.55	380	35
H	0.60	320	29

Root-node variability

The eight observed strengths are

$$56, 50, 54, 46, 44, 39, 35, 29.$$

The mean at the root node is

$$\acute{y}_{\text{root}} = \frac{56+50+54+46+44+39+35+29}{8} = 44.125\,\text{MPa}.$$

The root-node RSS is

$$\text{RSS}_{\text{root}} = \sum_{i=1}^{8} \left(y_i - 44.125\right)^2 = 614.875.$$

Candidate split 1: water-cement ratio ≤ 0.49

This split gives:

left child strengths: $56, 50, 54, 46, 44$;

right child strengths: $39, 35, 29$.

For the left child, the mean is

$$\acute{y}_L = \frac{56+50+54+46+44}{5} = 50.0\,\text{MPa}.$$

Its RSS is

$$\text{RSS}_L = (56-50)^2 + (50-50)^2 + (54-50)^2 + (46-50)^2 + (44-50)^2 = 104.$$

For the right child, the mean is

$$\acute{y}_R = \frac{39+35+29}{3} = 34.333\,\text{MPa}.$$

Its RSS is

$$\text{RSS}_R = (39-34.333)^2 + (35-34.333)^2 + (29-34.333)^2 \approx 50.667.$$

So the post-split RSS is

$$\text{RSS}_{\text{after}} = 104 + 50.667 = 154.667.$$

The RSS reduction is therefore

$$\Delta\,\text{RSS} = 614.875 - 154.667 = 460.208.$$

Candidate split 2: cement content $\leq 350\,\text{kg/m}^3$

This split gives:

left child strengths: $39, 29$;

right child strengths: $56, 50, 54, 46, 44, 35$.

For the left child,

$$\acute{y}_L = 34,\ \text{RSS}_L = (39-34)^2 + (29-34)^2 = 50.$$

For the right child, the mean is 47.5 MPa and the RSS is 291.5. Thus,

$$\text{RSS}_{\text{after}} = 50 + 291.5 = 341.5.$$

Because $154.667 < 341.5$, the split on water-cement ratio is clearly better.

Continuing the tree

Suppose the tree continues splitting on water-cement ratio.

In the left child, a split at 0.435 separates strengths $56, 50, 54$ from $46, 44$. The corresponding leaf means are 53.333 MPa and 45.0 MPa.

In the right child, a split at 0.575 separates strengths $39, 35$ from 29. The corresponding leaf means are 37.0 MPa and 29.0 MPa.

The resulting regression tree can therefore be read as:

if water-cement ratio is at most 0.435, predict 53.333 MPa;

if water-cement ratio is greater than 0.435 but at most 0.49, predict 45.0 MPa;

if water-cement ratio is greater than 0.49 but at most 0.575, predict 37.0 MPa;

if water-cement ratio exceeds 0.575, predict 29.0 MPa.

Final prediction for a new case

Suppose a new material sample has water-cement ratio 0.43 and cement content $430\,\mathrm{kg/m^3}$. The tree follows the first rule because $0.43 \le 0.435$. The predicted 28-day compressive strength is therefore approximately

$$\hat{y} = 53.333\,\mathrm{MPa}\,.$$

Engineering interpretation

This example reinforces a well-known engineering trend: lower water-cement ratio is often associated with higher strength. The tree captures that relationship as a sequence of thresholds rather than as a smooth formula. In a real data set, other variables such as curing temperature, aggregate properties, admixtures, or age could also appear in the tree. Even in this simplified case, the tree provides an immediately understandable design insight: the dominant variable in this data set is the water-cement ratio (Erdal, 2013).

> **Section summary.** The worked classification example shows how impurity reduction selects classifying rules, while the worked regression example shows how residual sum of squares selects predictive rules for a continuous engineering quantity. In both cases, the final value of the tree lies not only in prediction but also in the readability of the rule path.

8. Model understanding and practical interpretation

8.1 Strengths of decision trees

Decision trees offer several practical strengths.

Interpretability. A path from root to leaf can be read as a sequence of if-then rules. This supports engineering judgment, reporting, and communication with non-specialists.

Nonlinearity without a prescribed equation. The tree does not require the engineer to assume in advance that the relationship must be linear, quadratic, or of some other fixed algebraic form.

Interaction discovery. Trees can reveal that one variable matters differently in different ranges of another variable.

Mixed input types. Trees can work with both numerical and categorical predictors, though this tutorial has focused on numerical splits for mathematical clarity.

Minimal scaling requirements. Unlike some models, trees do not require all variables to be expressed on comparable numerical scales before the split logic makes sense.

These strengths explain why trees are often used as first models in engineering studies, especially when the analyst wants a transparent structure rather than only a numerical answer (Breiman et al., 1984; Loh, 2011).

8.2 Limitations of decision trees

A balanced understanding requires equal attention to limitations.

Overfitting risk. An unrestricted tree can become too detailed and start modelling noise rather than general structure.

Instability. Small changes in the data can sometimes change a split near the top of the tree, which in turn changes many downstream branches.

Biased split opportunities. Variables with many possible cut points may appear attractive simply because they offer more chances to produce an apparently good split.

Axis-aligned boundaries. Classical trees split one variable at a time, so diagonal or smoothly curved relationships may require many branches to approximate well.

Limited smoothness in regression. The regression surface is piecewise constant, which may be too abrupt for some engineering phenomena.

8.3 Situations where decision trees perform well

Decision trees are often particularly effective when:

the underlying engineering logic is threshold-based;

interpretability is as important as predictive performance;

the engineer wants an initial screening or triage tool;

interactions between variables are expected;

the problem includes both numerical and categorical information; or

the data are heterogeneous and difficult to summarise with one simple global equation.

Examples include condition screening, risk classification, maintenance prioritisation, and exploratory modelling in which the main goal is to learn the structure of the problem.

8.4 Conceptual comparison with other simple models

A brief conceptual comparison helps place decision trees in context.

Linear regression predicts a continuous output using a smooth linear combination of the inputs. It is compact and mathematically elegant, but it assumes a global equation form that may be too restrictive when threshold effects dominate.

Logistic regression is widely used for classification. It relates the class probability to the inputs through a smooth parametric formula. It is interpretable in coefficient form, but its interpretation is different from the rule-based interpretation of a tree.

Rule-based engineering heuristics are easy to communicate, but they are often created manually. Decision trees can be viewed as data-driven rule systems learned from examples.

The right choice depends on the purpose. If the main need is a transparent threshold rule, a decision tree is often highly attractive. If the phenomenon is smooth and approximately linear, a linear model may be more parsimonious.

8.5 Why decision trees are called interpretable

The word **interpretable** means that a human can understand how the model reached its conclusion. Decision trees are often considered interpretable because the production line from input to output is explicit. One can inspect the variables used near the top of the tree, the thresholds chosen, the class composition in leaves, or the mean responses in terminal regions.

Interpretability is especially valuable in Mechanical Engineering because decisions often affect safety, cost, maintenance priorities, and regulatory accountability. A model whose logic can be explained clearly is easier to scrutinise, challenge, and improve.

> **Section summary.** Decision trees are attractive because they are readable, flexible, and good at revealing thresholds and interactions. Their main weaknesses are overfitting, instability, biased split opportunities,

and limited smoothness in regression. Good engineering use therefore requires both appreciation of their strengths and awareness of their limits.

9. Beginner support and practical guidance

9.1 Common mistakes beginners make

Beginners often make the following mistakes when first learning decision trees.

Confusing inputs with outputs. The target variable must be clearly separated from the features.

Misidentifying the task type. A categorical target requires classification, while a continuous target requires regression.

Trusting training performance alone. Good fit on the training set is not enough; testing performance matters because the real goal is generalisation.

Allowing the tree to grow unchecked. A highly detailed tree may look impressive but can overfit badly.

Ignoring engineering meaning. A statistically chosen split should still be examined for physical plausibility and measurement reliability.

Treating association as causation. A tree can reveal predictive structure, but prediction alone does not prove a physical causal mechanism.

9.2 How to think about the data before thinking about the tree

Before studying the tree itself, students should first ask four practical questions.

What exactly is the prediction target?

What variables are available before the decision must be made?

Are the measurements reliable and meaningful in engineering terms?

What kind of decision will the prediction support?

These questions help prevent a common beginner error, namely focusing on the model before understanding the problem. In Mechanical Engineering, the usefulness of a model depends at least as much on the relevance of the variables and the quality of the measurements as on the mathematical algorithm.

9.3 Training, testing, underfitting, overfitting, and generalisation

These terms deserve one more clear interpretation.

Training is the phase in which the tree learns its split rules from labelled data.

Testing is the phase in which the completed tree is evaluated on unseen data.

Generalisation means successful performance on unseen data.

Underfitting means the tree is too simple and misses important structure.

Overfitting means the tree is too complex and captures noise as if it were signal.

A useful mental image is that underfitting draws too coarse a map, while overfitting draws every tiny crack and stain in the training sample. Good modelling seeks the middle ground.

9.4 A practical guide to deciding whether a problem is classification or regression

Table 7. Quick guide for deciding whether an engineering problem is classification or regression.

Question	If the answer is yes	Modelling type
Is the desired output a named category, such as safe/unsafe or good/fair/poor?	You are predicting labels	Classification
Is the desired output a measured quantity with units, such as MPa or mg/L?	You are predicting numbers	Regression
Do you mainly need class membership, screening groups, or risk categories?	The main goal is category assignment	Classification
Do you mainly need estimated magnitudes or continuous performance values?	The main goal is numerical estimation	Regression

If the output is a named category, the task is classification. If the output is a quantity with units, the task is regression. This simple rule already clarifies many engineering modelling questions.

9.5 Why trees feel intuitive, and why complexity control still matters

Decision trees feel intuitive because they resemble ordinary human reasoning: ask a question, split the possibilities, and continue. That is why they are excellent teaching models for students beginning ML. However, intuition should not be mistaken for automatic reliability. A tree can be easy to understand locally while still being statistically weak globally if it has been allowed to overfit.

For this reason, the best beginner habit is to pair interpretability with discipline:

define the modelling purpose clearly;

distinguish classification from regression correctly;

examine whether the chosen splits make engineering sense;

evaluate generalisation rather than training fit alone; and

control complexity through stopping rules and pruning.

> **Section summary.** The main beginner tasks are to identify the target correctly, separate classification from regression, understand the role of training and testing, and remember that intuitive models still require careful control of complexity.

10. Conclusion

Decision trees are among the most important introductory models in Machine Learning because they connect intuitive reasoning, mathematical structure, and practical interpretability. A tree predicts by recursively partitioning the feature space, creating regions that are increasingly homogeneous with respect to the target variable. In classification, homogeneity means class purity. In regression, it means low within-region variation.

For Mechanical Engineering students, this framework is especially valuable because many engineering decisions are naturally expressed through threshold-based rules. Material screening, material strength estimation, machined surface condition classification, structural health monitoring, bearing failure

susceptibility assessment, mechanics of materials and structures risk identification, and water-quality analysis are all examples of settings in which decision trees can provide useful and interpretable support.

The tutorial has emphasised four durable lessons. First, always distinguish clearly between classification and regression. Second, always connect the tree structure to its geometric meaning as a partition of the feature space. Third, always understand the mathematics of split quality, stopping, and pruning. Fourth, always interpret the model in engineering terms rather than accepting it as a black box.

With that mounting base in place, students are well prepared to study more advanced tree-based methods in the future. Even when more sophisticated models are later introduced, the conceptual lessons learned from decision trees remain essential: ask the right question, define the target carefully, control complexity, and interpret the result responsibly.

References

Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). *Classification and regression trees*. Wadsworth International Group.

Dvd8719. (2020a, June 22). Decision surface of classification tree. Wikimedia Commons.

Dvd8719. (2020b, June 22). Regression decision tree example. Wikimedia Commons.

Erdal, H. I. (2013). Two-level and hybrid ensembles of decision trees for high performance mechanical material strength prediction. Engineering Applications of Artificial Intelligence, 26(7), 1689–1697.

Gandomi, A. H., Fridline, M. M., & Roke, D. A. (2013). Decision tree approach for material liquefaction assessment. *The Scientific World Journal*, *2013*, 346285.

Gilgoldm. (2020, May 18). Decision tree - survival of passengers on the titanic. Wikimedia Commons.

Glinscaya, A., Panfilov, I., Kukartsev, A., Suprun, E., & Boyko, A. (2024). Use of decision trees for coolant quality assessment: Analysis of key parameters. BIO Web of Conferences, 130, 03002.

Gomez-Cabrera, A., & Escamilla-Ambrosio, P. J. (2022). Review of machine-learning techniques applied to structural health monitoring systems for building and mechanical component structures. Applied Sciences, 12(21), 10754.

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer.

James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An introduction to statistical learning: With applications in r* (2nd ed.). Springer.

Loh, W.-Y. (2011). Classification and regression trees. Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery, 1(1), 14–23.

Mitchell, T. M. (1997). *Machine learning*. McGraw-Hill.

Mondal, T. G., & Chen, G. (2022). Artificial intelligence in mechanical mechanical systems health monitoring: Historical perspectives, current trends, and future visions. Frontiers in Built Environment, 8, 1007886.

Nefeslioglu, H. A., Sezer, E., Gokceoglu, C., Bozkir, A. S., & Duman, T. Y. (2010). Assessment of bearing failure susceptibility by decision trees in the metropolitan area of istanbul, turkey. Mathematical Problems in Engineering, 2010, 901095.

Quinlan, J. R. (1986). Induction of decision trees. Machine Learning, 1, 81–106.

Rossc0827. (2016, December 13). Recursive splitting. Wikimedia Commons.

Russell, S., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson.

Shao, W., Yue, W., Zhang, Y., Zhou, T., Zhang, Y., Dang, Y., Wang, H., Feng, X., & Chao, Z. (2023). The application of machine learning techniques in mechanics of materials and structures: A review and comparison. Mathematics, 11(18), 3976.

Tamagusko, T., Correia, M. G., & Ferreira, A. (2024). Machine learning applications in production line machined surface management: A review, challenges and future directions. Mechanical systems, 9(12), 213.